Homework · Practice What You Learned

Write your answers in new cells below.

Homework 1 · Embed your own knowledge base *(Embeddings, dimension cost)*

1. Write 10 short sentences about a topic you know well (a hobby, a city, a sport). Give each one a `topic` label.
2. Embed them with `embed_documents()` and print the number of vectors and the dimension of each.
3. Calculate how much memory 5 million of your vectors would need in float32 (`dim × 4 bytes`).
4. Request the same text with `dimensions=256` from `openai/text-embedding-3-large`. How much memory do you save, and what do you give up?

Goal: see that an embedding is just a fixed-length list of floats, and that its size drives storage cost.

Homework 2 · Compare distance metrics *(Cosine, Euclidean, Dot product)*

1. Embed three queries of your own: one that closely matches a document, one that is loosely related, and one that is completely unrelated.
2. For each query, compute cosine similarity, Euclidean distance and dot product against all your documents (`cosine_similarity`, `euclidean_distances`, `linear_kernel`).
3. Print the top 3 documents per metric. Do the rankings agree?
4. Check whether your vectors are unit length with `np.linalg.norm`. Explain why the rankings do or do not match, using `d² = 2 − 2·cos`.

Goal: understand when the three metrics agree, and why normalized

vectors make them equivalent.

Homework 3 · Build your first Qdrant collection *(Collection, points, search)*

1. Create a new collection with the correct `size` and `Distance.COSINE`.
2. Upsert your documents as points with a payload of `text` and `topic`.
3. Search with your three queries from Homework 2 and print `score`, `id` and `text`.
4. Re-upsert one point with the same `id` but different text. Confirm the count did not grow (`qdrant.count(...)`), and explain what "upsert" did.
5. Check that Qdrant's ranking matches your manual `retrieve()` function.

Goal: get comfortable with the collection → point → payload model and verify Qdrant returns what the maths predicts.

Homework 4 · Exact vs HNSW and Recall@K *(Search strategies)*

1. Duplicate your documents with small text variations until you have at least 200 points. Embed them in batches.
2. Create two collections: one searched with `exact=True`, one with an HNSW index (`m=8` and `m=32`, `ef_construct=100`).
3. For 10 queries, compute Recall@5 of each HNSW setup against the exact results while varying `hnsw_ef` in `[8, 16, 32, 64, 128]`.
4. Record latency for each setting and plot recall vs latency (a simple table is fine).
5. Write two sentences: which `m` / `hnsw_ef` would you pick for production, and why?

Goal: treat exact search as ground truth, measure what approximate search loses, and tune the recall/latency trade-off.

Homework 5 · Quantization and filtering *(Compression, payload filters, final comparison)*

1. Create an INT8 scalar-quantized collection and try `always_ram=True` and `always_ram=False`. Compare recall against the exact collection.
2. Add a `level` payload field if you do not have one, and create payload indexes on `topic` and `level`.
3. Run the same query with: no filter, `topic == "ai"`, and `topic == "ai"` **and** `level != "beginner"` (`must` + `must_not`). Show how the results change.
4. Build one summary table for your query set with the columns **Strategy · Recall@5 · Avg latency · Memory estimate** and the rows **Exact, HNSW, HNSW + INT8, HNSW + filter**.
5. In three bullet points, recommend a strategy for (a) 10 thousand documents, (b) 100 million documents, (c) a multi-tenant SaaS product.

Goal: bring every concept together and justify a search strategy with measurements rather than guesses.

> **Stretch goal:** wrap your best configuration in a `rag_retrieve(question)` function and test it on five questions you wrote yourself